

MECEE4606 Digital Manufacturing

Assignment 1 - Laser Cut Vessel



Names: Wei Sun, Lei Cui

UNIs: ws2782, lc4028

Date/Time submitted: 2026/2/4 7p.m.

Grace hour: 96h(before) + 125(gained) = 221h(After)

This small rectangle had submitted in Jan 29th, a week before deadline.

Small rectangle sample

Image/text size and location matching

```
scale_factor = 0.001 / width_in
image_width = 1264 * scale_factor
image_height = 1028 * scale_factor
translate_x = (width_in - image_width) / 2
translate_y = (height_in - image_height) / 2

# Calculate text position - ws2782
text_y = translate_y + image_height + 0.2
text_x = width_in / 2
```

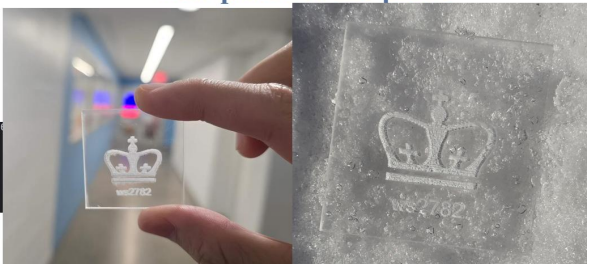
Input parameters

```
C:\Users\Lenovo\PysparkProjects\MECE4606\venv\Scripts\python.exe
Enter rectangle width (inches, ≤ 2): 1.5
Enter rectangle height (inches, ≤ 2): 1.5
SVG file 'rectangle_CU_ws2782.svg' generated successfully.
```

SVG file

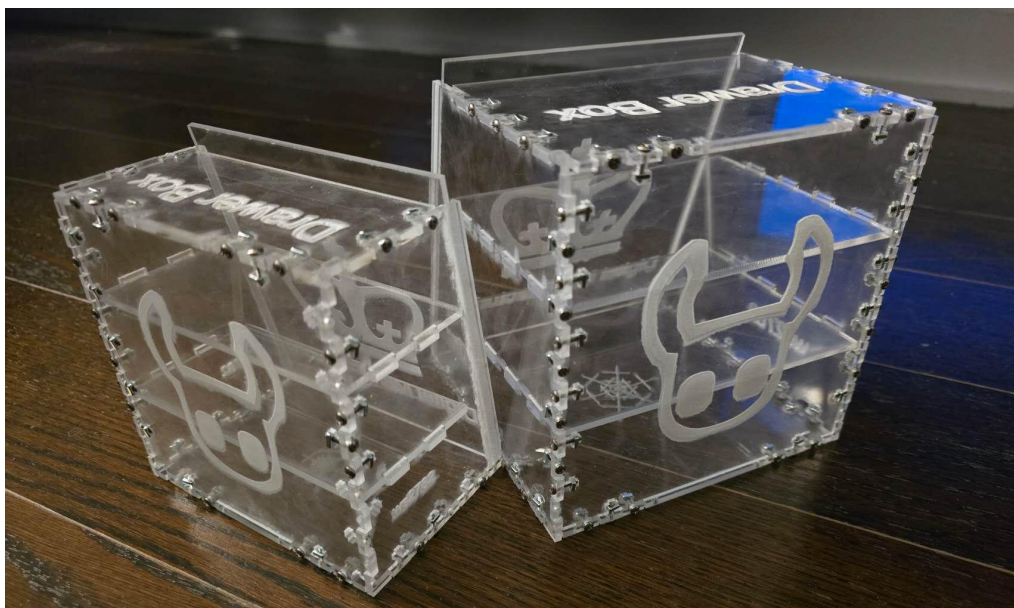
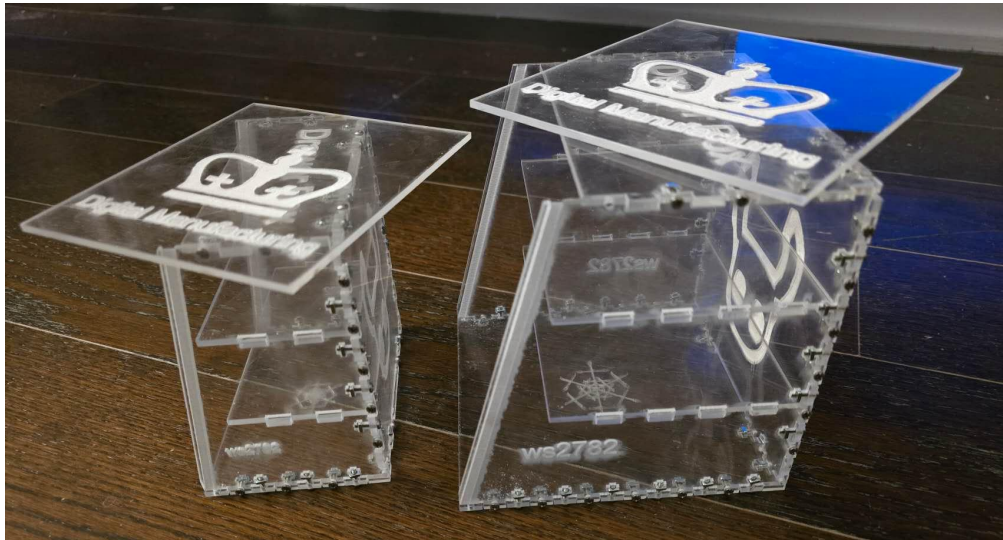


Real photo of the piece



The box is called Drawer Box. This box draws inspiration from sliding drawers, where the tilted lid slides freely along the side panel's grooves, enabling easy opening and closing. The detailed pictures are given below. The left one is 120mm * 80mm * 80mm while right one is 140mm * 140mm * 140mm.

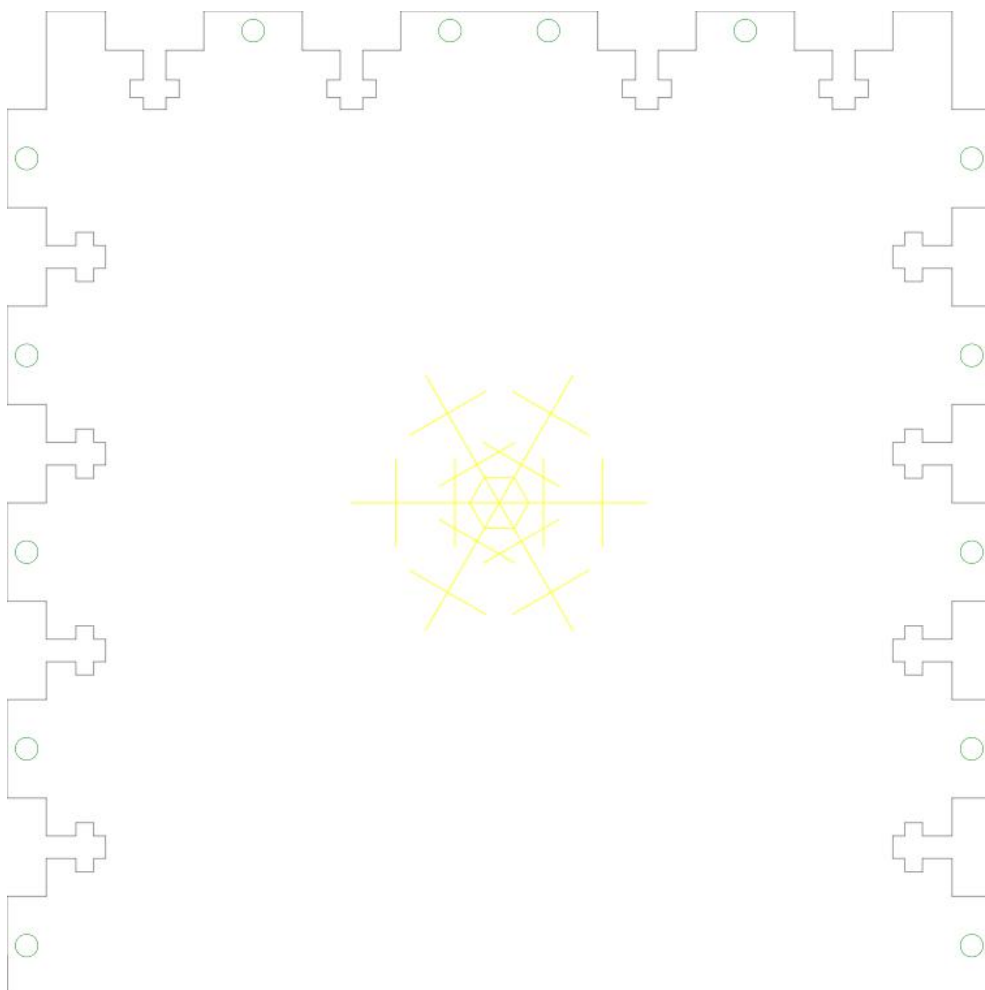




Software inputs of box and detects errors:

```
Drawer Box Part1 x
C:\Users\Lenovo\PycharmProjects\MECEE4606\.venv\Scripts\pyt
Enter rectangle width (millimeters): 100
Enter rectangle height (millimeters): 100
SVG file 'Drawer Box part1.svg' generated successfully.
Snowflake added at center position: (50.0, 50.0)
Snowflake size: 15.0mm
```

Take the bottom plane as example, when the python code runs, the height and width need to be entered to generate the svg figures. The generated figure is shown below:



However, if the input is less than the given limited number, it will give the error feedback like:

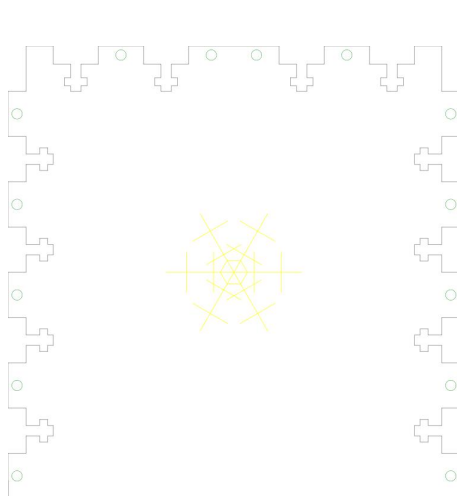
```
C:\Users\Lenovo\PycharmProjects\MECEE4606\.venv\Scr
Enter rectangle width (millimeters): 50
Enter rectangle height (millimeters): 50
Error: Width and height must be more than 60mm.
```

Or

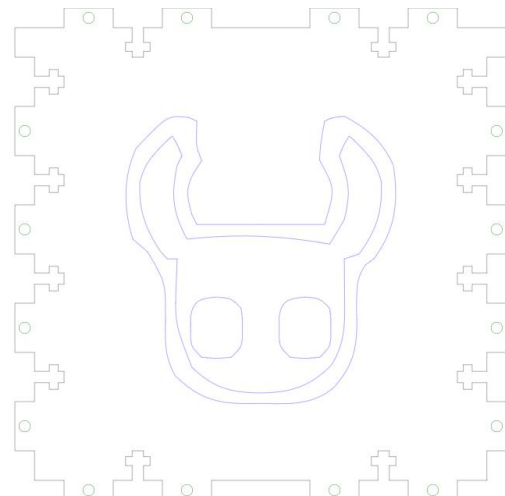
```
C:\Users\Lenovo\PycharmProjects\MECEE4606\.venv\Scr
Enter rectangle width (millimeters): 200
Enter rectangle height (millimeters): 200
Error: Width and height must be 150mm or smaller.
```

Here are four types of the box svg figure I generated with different parameters(the length and height of the box are same due to the structure):

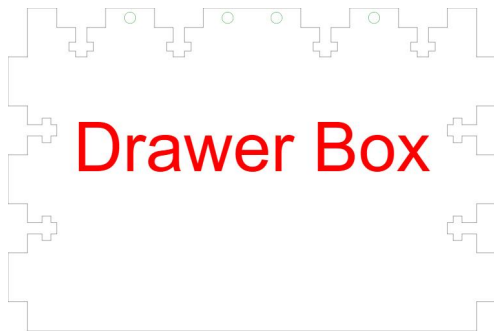
1. 100mm * 100mm * 100mm(w, h , l)



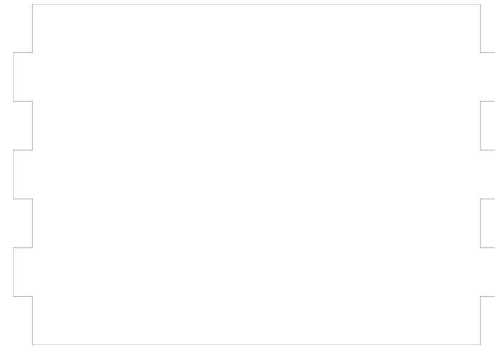
Bottom Plane



Back Plane



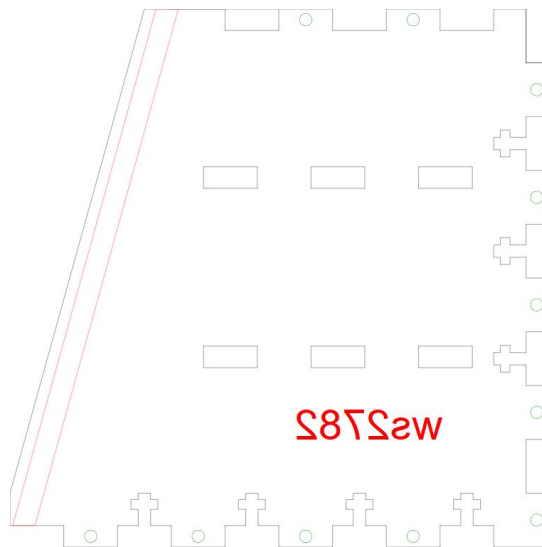
Top plane



Middle divider * 2

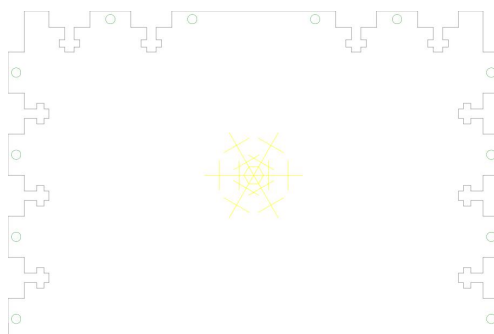


Lid

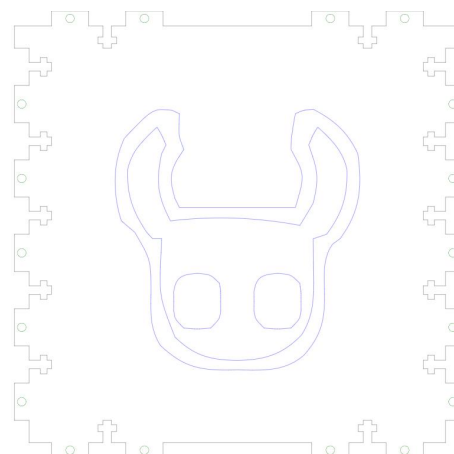


Left side plane(right one is inverse)

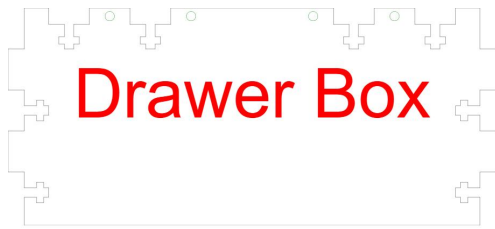
2. 120mm * 80mm * 80mm(the parameters I used to generate left box in picture)



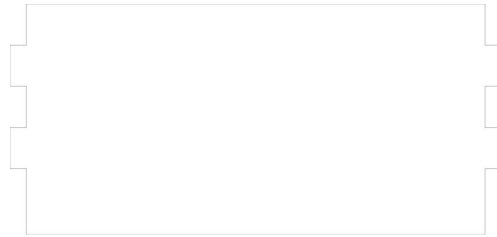
Bottom Plane



Back Plane



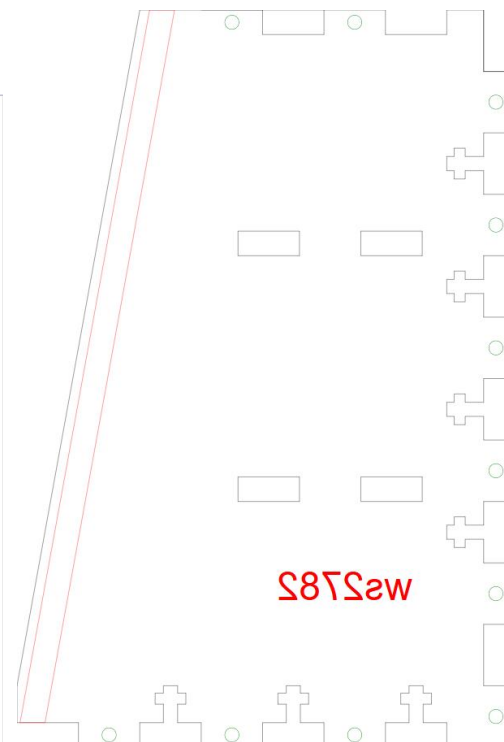
Top plane



Middle divider * 2

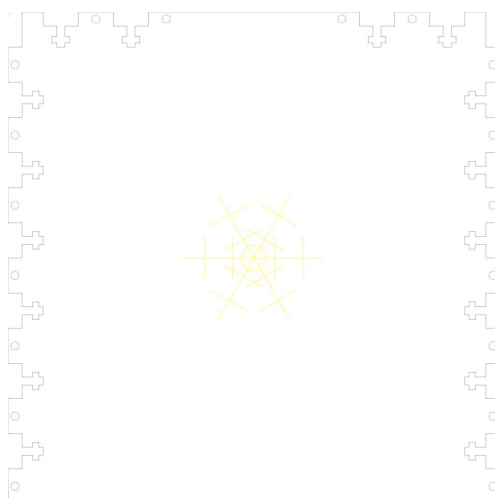


Lid

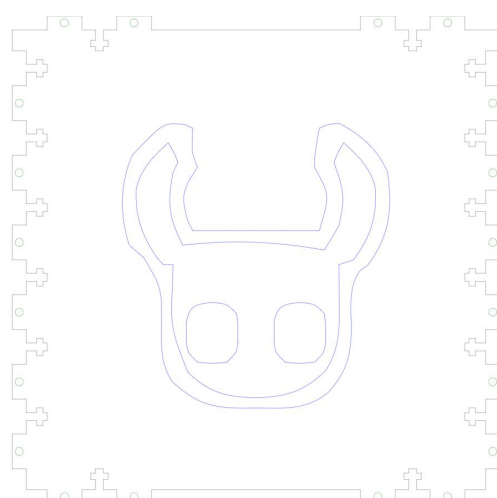


Left side plane

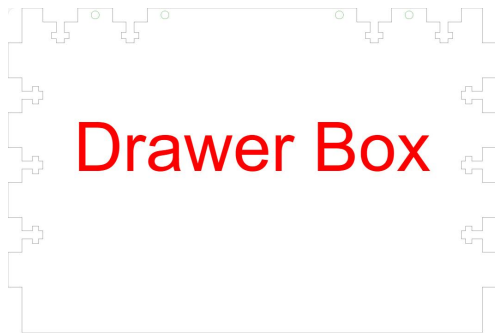
3. 140mm * 140mm * 140mm(parameters for right box)



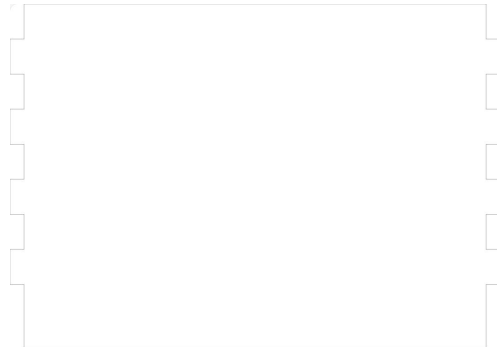
Bottom Plane



Back Plane



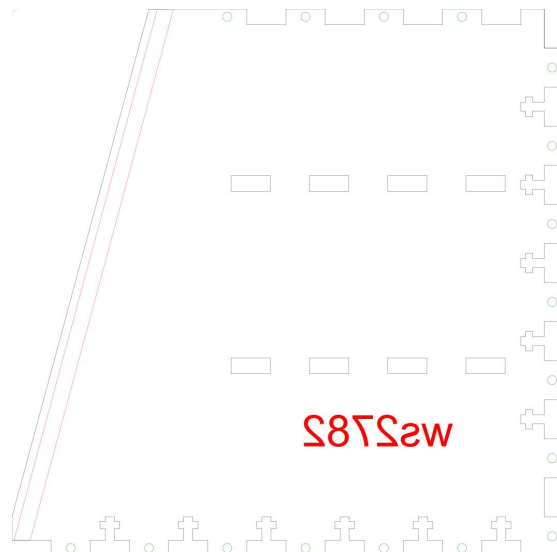
Top plane



Middle divider * 2

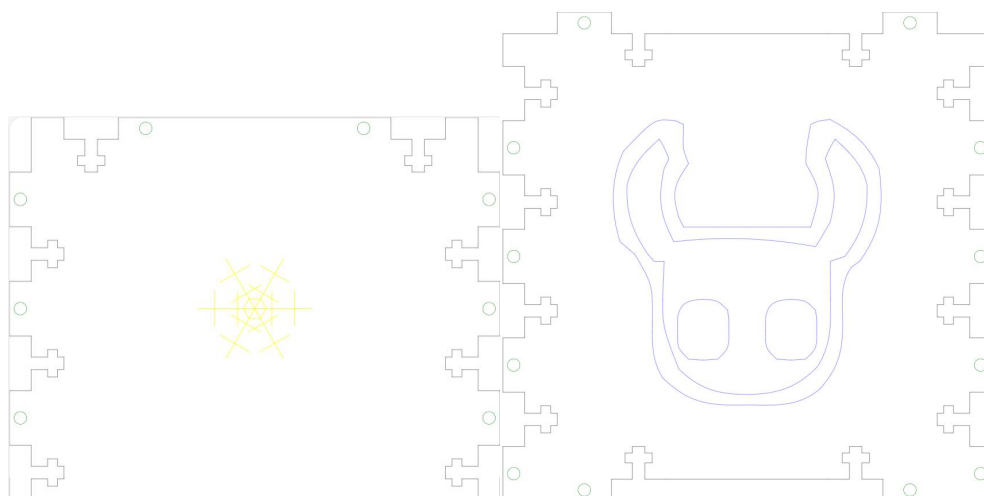


Lid



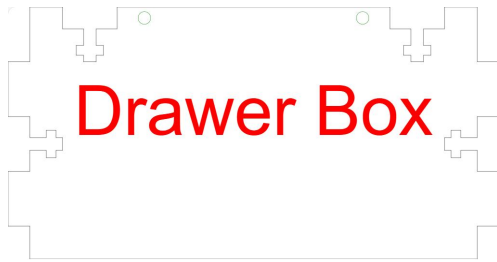
Left side plane

4. 90mm * 70mm * 70mm



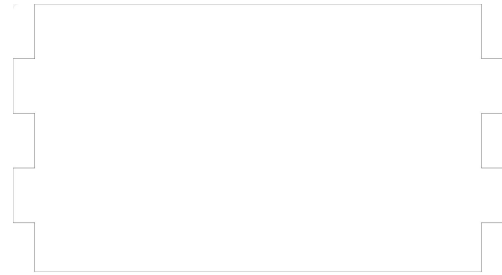
Bottom Plane

Back Plane



Drawer Box

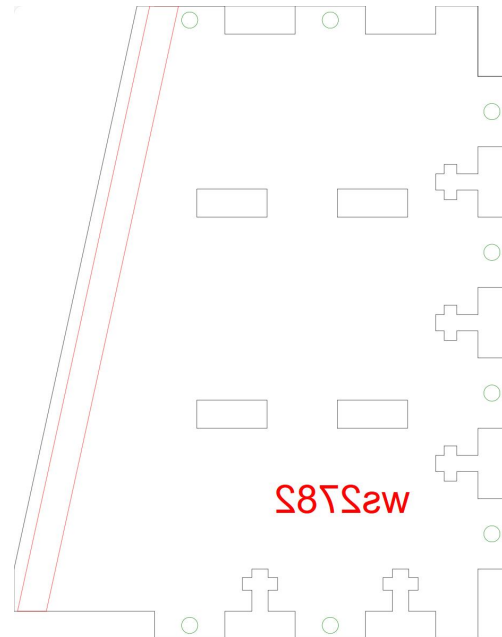
Top plane



Middle divider * 2



Lid



Left side plane

This time, take the bottom plane(part1)'s code as example, give the description of the software I wrote – calculation steps, formulas, conditions.

SVG Drawer Box Part Generator with Snowflake Decoration.

Overview

This Python software generates an SVG file representing a drawer box component with decorative snowflake patterns. The program creates technical drawings suitable for laser cutting or CNC machining, featuring assembly notches, screw holes, and an artistic snowflake at the center.

Core Parameters & Conditions

- Input Constraints:

Width: $60\text{mm} < w \leq 150\text{mm}$

Height: $60\text{mm} < h \leq 150\text{mm}$

- Fixed Dimensions:

Notch height: 4mm

Notch width: 10mm

Screw diameter: 2.3mm

Screw length: 10mm

Nut dimensions: 1.8mm × 5mm

- Notch Positioning:

Width < 95mm: 2 notches at positions 10mm and (width - 20)mm

Width $\geq 95\text{mm}$: 4 notches at 10mm, 30mm, (width - 40)mm, and (width - 20)mm

Key Calculation Steps

1. Coordinate System & Unit Conversion

SVG ViewBox: (0, 0, width_mm, height_mm)

Output units: Inches (width_mm/25.4, height_mm/25.4)

2. Main Perimeter Path Generation

- Horizontal top edge with notches:

For each notch_position x:

If current_x < x: Draw line from current_x to x at y=0

Draw notch structure at (x, 0)

Update current_x = x + notch_width

- Vertical side patterns:

y = 0

While y + 10 < height_mm:

Left side: Line from (0, y+10) to (0, min(y+20, height_mm))

Right side: Line from (width_mm, y+10) to (width_mm, min(y+20, height_mm))

y += (notch_width + 10)

- Bottom edge: Straight line from (0, height_mm) to (width_mm,

height_mm)

3. Notch Geometry Calculations

- Each notch includes screw and nut cutouts with precise dimensional relationships:

Screw inset: screw_in = screw_length - notch_height = 6mm

Nut inset: nut_in = 3mm

Upper notch structure:

Vertical: 0 $\rightarrow$ notch_height

Horizontal: Various segments for screw clearance and nut pocket

Dimensions derived from: (notch_width $\pm$ screw_d)/2, (notch_width $\pm$ nut_w)/2

4. Side Notch Pattern Generation

- For left and right sides at vertical intervals:

If $y + \text{notch_width} \leq 20\text{mm}$: Simple rectangular notch

Else: Complex notch with screw/nut cutouts

Vertical segments at: y , $y+(\text{notch_width} \pm \text{screw_d})/2$, $y+(\text{notch_width} \pm \text{nut_w})/2$

Horizontal offsets: notch_height , $\text{notch_height} + \text{nut_in}$, $\text{notch_height} + \text{nut_in} + \text{nut_l}$

5. Snowflake Generation Algorithm

Parameters:

$\text{outer_radius} = \text{snowflake_size}$ (15% of $\min(\text{width}, \text{height})$, clamped 10-20mm)

$\text{inner_radius} = \text{outer_radius} \times 0.6$

$\text{arm_count} = 6$

$\text{small_arms_per_arm} = 2$

Geometry:

Main arms: Center to outer_radius at 60° intervals ($2\pi/\text{arm_count}$)

Small arms: Positioned at 30% and 70% along each main arm

Small arm angles: $\pm 90^\circ$ from main arm ($\text{angle} \pm \pi/2$)

Small arm length: $\text{inner_radius} \times 0.5$

Central hexagon: $\text{Radius} = \text{snowflake_size} \times 0.2$

6. SVG Path Construction

- Uses M (move), L (line), and Z (close path) commands
- Numerical precision: 2 decimal places
- Multiple path segments concatenated with spaces

7. SVG Assembly

Three visual layers:

- Black strokes (0.05mm): Main box structure
- Green circles (0.05mm): Screw holes
- Yellow snowflake (0.1mm): Decorative element with rounded line caps

Mathematical Formulas

1. Polar to Cartesian Conversion:

$$x = \text{center_x} + \text{radius} \times \cos(\text{angle})$$

$y = \text{center_y} + \text{radius} \times \sin(\text{angle})$

2.Linear Interpolation for Small Arm Placement:

$\text{arm_x} = \text{start_x} + t \times (\text{end_x} - \text{start_x})$

$\text{arm_y} = \text{start_y} + t \times (\text{end_y} - \text{start_y})$

where $t \in \{0.3, 0.7\}$

3.Snowflake Size Calculation:

$\text{snowflake_size} = \text{clamp}(\min(\text{width_mm}, \text{height_mm}) \times 0.15, 10, 20)$

4.Hexagon Vertices:

For $i = 0$ to 5 :

$\text{angle} = i \times 2\pi / 6$

$\text{vertex} = \text{center} + (\text{snowflake_size} \times 0.2) \times (\cos(\text{angle}), \sin(\text{angle}))$

Error Handling

- Validates input dimensions against constraints
- Provides clear error messages for invalid inputs
- Ensures generated geometry fits within specified bounds

Output

- SVG file with proper XML declaration and namespaces
- Console confirmation with snowflake position and size
- Technical drawing ready for manufacturing processes

References

- [1] J. Blutinger, *DM01_2DCutting*, Digital Manufacturing (MECE 4606), Columbia University, Spring 2026. [3-54].

Appendix

Here are the python code for generating the svg figures.

1. Bottom plane(part1) - Drawer Box Part 1.py

```
import math

def generate_snowflake_path(center_x, center_y, size=10):

    outer_radius = size
    inner_radius = size * 0.6
    arm_count = 6
    small_arms_per_arm = 2

    path_d = []

    for i in range(arm_count):
        angle = (i * 2 * math.pi) / arm_count

        start_x = center_x
        start_y = center_y

        end_x = center_x + outer_radius * math.cos(angle)
        end_y = center_y + outer_radius * math.sin(angle)

        path_d.append(f"M {start_x:.2f} {start_y:.2f}")
        path_d.append(f"L {end_x:.2f} {end_y:.2f}")

        for j in range(small_arms_per_arm):
            t = 0.3 + 0.4 * j
            arm_x = start_x + t * (end_x - start_x)
            arm_y = start_y + t * (end_y - start_y)

            small_arm_angle1 = angle + math.pi / 2
            small_arm_angle2 = angle - math.pi / 2

            small_arm_length = inner_radius * 0.5

            small_end_x1 = arm_x + small_arm_length * math.cos(small_arm_angle1)
            small_end_y1 = arm_y + small_arm_length * math.sin(small_arm_angle1)
            path_d.append(f"M {arm_x:.2f} {arm_y:.2f}")
            path_d.append(f"L {small_end_x1:.2f} {small_end_y1:.2f}")

            small_end_x2 = arm_x + small_arm_length * math.cos(small_arm_angle2)
```

```

        small_end_y2 = arm_y + small_arm_length * math.sin(small_arm_angle2)
        path_d.append(f"M {arm_x:.2f} {arm_y:.2f}")
        path_d.append(f"L {small_end_x2:.2f} {small_end_y2:.2f}")

hexagon_points = []
for i in range(6):
    angle = (i * 2 * math.pi) / 6
    hx = center_x + (size * 0.2) * math.cos(angle)
    hy = center_y + (size * 0.2) * math.sin(angle)
    hexagon_points.append(f"{hx:.2f},{hy:.2f}")

path_d.append(f"M {hexagon_points[0]}")
for i in range(1, 6):
    path_d.append(f"L {hexagon_points[i]}")
path_d.append("Z")

return " ".join(path_d)

def generate_Part1_svg(width_mm, height_mm, filename="Drawer Box part1.svg"):
    if width_mm <= 60 or height_mm <= 60:
        raise ValueError("Width and height must be more than 60mm.")

    if width_mm > 150 or height_mm > 150:
        raise ValueError("Width and height must be 150mm or smaller.")

    notch_height = 4
    notch_width = 10
    screw_d = 2.3
    screw_l = 10
    screw_in = screw_l - notch_height
    nut_l = 1.8
    nut_w = 5
    nut_in = 3

    if width_mm < 95:
        notch_positions = [10, width_mm - 20]
    else:
        notch_positions = [10, 30, width_mm - 40, width_mm - 20]

    width_in = width_mm / 25.4
    height_in = height_mm / 25.4
    path_d = ""
    circles_svg = ""

```

```

# ===== SVG Path =====
current_x = notch_height
for x in notch_positions:
    if x > current_x:
        path_d += f"M {current_x} 0 H {x}\n"
        current_x = x + notch_width

if current_x < width_mm:
    path_d += f"M {current_x} 0 H {width_mm - notch_height}\n"

y = 0
step = notch_width + 10

while y + 10 < height_mm:
    path_d += f"M 0 {y + 10} V {min(y + 20, height_mm)}\n"
    y += step

y = 0
while y + 10 < height_mm:
    path_d += f"M {width_mm} {y + 10} V {min(y + 20, height_mm)}\n"
    y += step

path_d += f"M 0 {height_mm} H {width_mm}\n"

# up rect
for x in notch_positions:
    path_d += f""
    M {x} 0
    V {notch_height}
    H {x + (notch_width - screw_d) / 2}
    V {notch_height + nut_in}
    H {x + (notch_width - nut_w) / 2}
    V {notch_height + nut_in + nut_l}
    H {x + (notch_width - screw_d) / 2}
    V {notch_width}
    H {x + (notch_width + screw_d) / 2}
    V {notch_height + nut_in + nut_l}
    H {x + (notch_width + nut_w) / 2}
    V {notch_height + nut_in}
    H {x + (notch_width + screw_d) / 2}
    V {notch_height}
    H {x + notch_width}
    V {0}

```

```

"""
if x < width_mm / 2:
    circles_svg += f"""
        <circle cx="{x + notch_width / 2 + 10}"
            cy="{notch_height / 2}"
            r="{screw_d / 2}" />
    """
elif x > width_mm / 2:
    circles_svg += f"""
        <circle cx="{x + notch_width / 2 - 10}"
            cy="{notch_height / 2}"
            r="{screw_d / 2}" />
    """

# ===== side rect =====
y = 0
step = notch_width + 10

path_d += f"""
    M 0 {width_mm}
    V {width_mm - width_mm % 10}
"""

path_d += f"""
    M {width_mm} {width_mm}
    V {width_mm - width_mm % 10}
"""

while y < height_mm:
    # left
    if y + notch_width <= 20:
        path_d += f"""
            M {notch_height} {y}
            V {y + notch_width}
            H 0
        """
    elif y + notch_width <= height_mm:
        path_d += f"""
            M 0 {y}
            H {notch_height}
            V {y + (notch_width - screw_d) / 2}
            H {notch_height + nut_in}
            V {y + (notch_width - nut_w) / 2}
            H {notch_height + nut_in + nut_l}
            V {y + (notch_width - screw_d) / 2}
        """

```

```

H {notch_width}
V {y + (notch_width + screw_d) / 2}
H {notch_height + nut_in + nut_l}
V {y + (notch_width + nut_w) / 2}
H {notch_height + nut_in}
V {y + (notch_width + screw_d) / 2}
H {notch_height}
V {y + notch_width}
H 0
"""
circles_svg += f"""
    <circle cx="{notch_height / 2}"
           cy="{y + notch_width / 2 + 10}"
           r="{screw_d / 2}" />
    """

# right
if y + notch_width <= 20:
    path_d += f"""
    M {width_mm - notch_height} {y}
    V {y + notch_width}
    H {width_mm}
    """
elif y + notch_width <= height_mm:
    path_d += f"""
    M {width_mm} {y}
    H {width_mm - notch_height}
    V {y + (notch_width - screw_d) / 2}
    H {width_mm - (notch_height + nut_in)}
    V {y + (notch_width - nut_w) / 2}
    H {width_mm - (notch_height + nut_in + nut_l)}
    V {y + (notch_width - screw_d) / 2}
    H {width_mm - notch_width}
    V {y + (notch_width + screw_d) / 2}
    H {width_mm - (notch_height + nut_in + nut_l)}
    V {y + (notch_width + nut_w) / 2}
    H {width_mm - (notch_height + nut_in)}
    V {y + (notch_width + screw_d) / 2}
    H {width_mm - notch_height}
    V {y + notch_width}
    H {width_mm}
    """
circles_svg += f"""
    <circle cx="{width_mm - notch_height / 2}"

```

```

        cy="{y + notch_width / 2 + 10}"
        r="{screw_d / 2}" />
    ""

    y += step

    path_d += f""
    M 0 {height_mm - notch_height}
    V {height_mm}
    ""

    path_d += f""
    M {width_mm} {height_mm - notch_height}
    V {height_mm}
    ""

    # ===== snow figure generate =====
    # size
    snowflake_size = min(width_mm, height_mm) * 0.15
    snowflake_size = max(10, min(snowflake_size, 20))

    # center
    snowflake_center_x = width_mm / 2
    snowflake_center_y = height_mm / 2

    # path of snow
    snowflake_path = generate_snowflake_path(
        snowflake_center_x,
        snowflake_center_y,
        snowflake_size
    )

    svg_content = f'<?xml version="1.0" standalone="no"?>
<svg xmlns="http://www.w3.org/2000/svg"
    width="{width_in}in"
    height="{height_in}in"
    viewBox="0 0 {width_mm} {height_mm}">

<path d="{path_d}"
    fill="none"
    stroke="black"
    stroke-width="0.05"
    fill-rule="evenodd"/>

```

```

<g fill="none" stroke="green" stroke-width="0.05">
    {circles_svg}
</g>

<path d="{snowflake_path}"
      fill="none"
      stroke="yellow"
      stroke-width="0.1"
      stroke-linecap="round"/>

</svg>
'''

with open(filename, "w", encoding="utf-8") as f:
    f.write(svg_content)

print(f"SVG file '{filename}' generated successfully.")
print(f"Snowflake added at center position: ({snowflake_center_x:.1f}, {snowflake_center_y:.1f})")
print(f"Snowflake size: {snowflake_size:.1f}mm")

if __name__ == "__main__":
    try:
        w = float(input("Enter rectangle width (millimeters): "))
        h = float(input("Enter rectangle height (millimeters): "))

        generate_Part1_svg(w, h)

    except ValueError as e:
        print("Error:", e)

```

2. Back plane(part2) - Drawer Box Part 2.py

```
def generate_Part2_svg(width_mm, filename="Drawer Box part2.svg"):

    if width_mm <= 60:
        raise ValueError("Width must be more than 60mm.")

    if width_mm > 150:
        raise ValueError("Width must be 150mm or smaller.")

    notch_height = 4
    notch_width = 10
    screw_d = 2.3
    screw_l = 10
    screw_in = screw_l - notch_height
    nut_l = 1.8
    nut_w = 5
    nut_in = 3
    embed_w = 64
    embed_h = 64
    center_x = width_mm / 2
    center_y = width_mm / 2
    scale = width_mm / 100
    if width_mm < 95:
        notch_positions = [10, width_mm - 20]
    else:
        notch_positions = [10, 30, width_mm - 40, width_mm - 20]
    height_mm = width_mm
    width_in = width_mm / 25.4
    height_in = width_mm / 25.4
    path_d = ""
    circles_svg = ""
    # ===== SVG Path =====
    # up and down lines
    current_x = 0
    for x in notch_positions:
        path_d += f"M {x} 0 H {x + notch_width}\n"
        path_d += f"M {x} {width_mm} H {x + notch_width}\n"
        current_x = x + notch_width

    # left lines
    y = 0
    step = notch_width + 10
    while y < height_mm:
```

```

    if y < notch_height:
        path_d += f"M 0 {notch_height} V {notch_width}\n"
        y += step
    elif y < height_mm:
        path_d += f"M 0 {y} V {min(y + 10, height_mm)}\n"
        y += step

# right lines
y = 0
while y < height_mm:
    if y < notch_height:
        path_d += f"M {width_mm} {notch_height} V {notch_width}\n"
        y += step
    elif y < height_mm:
        path_d += f"M {width_mm} {y} V {min(y + 10, height_mm)}\n"
        y += step

# up and down gap
if width_mm < 95:
    start_x = 30
    end_x = width_mm - 30
else:
    start_x = 40
    end_x = width_mm - 40

if width_mm < 95:
    path_d += f"""\n
M {start_x} {notch_height}
H {end_x}
"""\n
    path_d += f"""\n
M {start_x} {width_mm - notch_height}
H {end_x}
"""\n
else:
    path_d += f"""\n
M {start_x} 0
V {notch_height}
H {end_x}
V 0
"""\n
    path_d += f"""\n
M {start_x} {width_mm}
V {width_mm - notch_height}
H {end_x}
"""

```

```

        V {width_mm}
        """"
path_d += f""""
    M 0 {notch_height}
    H {notch_width}
    V 0
    """"

path_d += f""""
    M {width_mm - 10} 0
    V {notch_height}
    H {width_mm}
    """"

# up rect
x_positions = [20, width_mm - 30]
for x in x_positions:
    if width_mm < 95:
        path_d += f""""
            M {x} {notch_height}
            H {x + (notch_width - screw_d) / 2}
            V {notch_height + nut_in}
            H {x + (notch_width - nut_w) / 2}
            V {notch_height + nut_in + nut_l}
            H {x + (notch_width - screw_d) / 2}
            V {notch_width}
            H {x + (notch_width + screw_d) / 2}
            V {notch_height + nut_in + nut_l}
            H {x + (notch_width + nut_w) / 2}
            V {notch_height + nut_in}
            H {x + (notch_width + screw_d) / 2}
            V {notch_height}
            H {x + notch_width}
            """"

        if x < width_mm / 2:
            circles_svg += f""
                <circle cx="{x + notch_width / 2 - 10}"
                    cy="{notch_height / 2}"
                    r="{screw_d / 2}" />
                ""

        elif x > width_mm / 2:
            circles_svg += f""
                <circle cx="{x + notch_width / 2 + 10}"
                    cy="{notch_height / 2}"
                    r="{screw_d / 2}" />
            ""

```

```

'''

else:
    path_d += f""
    M {x} 0
    V {notch_height}
    H {x + (notch_width - screw_d) / 2}
    V {notch_height + nut_in}
    H {x + (notch_width - nut_w) / 2}
    V {notch_height + nut_in + nut_l}
    H {x + (notch_width - screw_d) / 2}
    V {notch_width}
    H {x + (notch_width + screw_d) / 2}
    V {notch_height + nut_in + nut_l}
    H {x + (notch_width + nut_w) / 2}
    V {notch_height + nut_in}
    H {x + (notch_width + screw_d) / 2}
    V {notch_height}
    H {x + notch_width}
    V 0
    ""

    if x < width_mm / 2:
        circles_svg += f""
        <circle cx="{x + notch_width / 2 + 10}"
            cy="{notch_height / 2}"
            r="{screw_d / 2}" />
        ""
        circles_svg += f""
        <circle cx="{x + notch_width / 2 - 10}"
            cy="{notch_height / 2}"
            r="{screw_d / 2}" />
        ""

    elif x > width_mm / 2:
        circles_svg += f""
        <circle cx="{x + notch_width / 2 - 10}"
            cy="{notch_height / 2}"
            r="{screw_d / 2}" />
        ""
        circles_svg += f""
        <circle cx="{x + notch_width / 2 + 10}"
            cy="{notch_height / 2}"
            r="{screw_d / 2}" />
        ""

```

```

# make up down gap
if width_mm % 20 <= 10 and not width_mm % 20 == 0:
    path_d += f""
        M 0 {width_mm}
        H {notch_width}
        ""

    path_d += f""
        M {width_mm - notch_width} {width_mm}
        H {width_mm}
        ""

elif width_mm % 20 > 10 and not width_mm % 20 == 0:
    path_d += f""
        M {notch_height} {width_mm}
        H {notch_width}
        ""

    path_d += f""
        M {width_mm - notch_height} {width_mm}
        H {width_mm - notch_width}
        ""

#down rect
for x in x_positions:
    if width_mm < 95:
        path_d += f""
            M {x} {width_mm - notch_height}
            H {x + (notch_width - screw_d) / 2}
            V {width_mm - (notch_height + nut_in)}
            H {x + (notch_width - nut_w) / 2}
            V {width_mm - (notch_height + nut_in + nut_l)}
            H {x + (notch_width - screw_d) / 2}
            V {width_mm - notch_width}
            H {x + (notch_width + screw_d) / 2}
            V {width_mm - (notch_height + nut_in + nut_l)}
            H {x + (notch_width + nut_w) / 2}
            V {width_mm - (notch_height + nut_in)}
            H {x + (notch_width + screw_d) / 2}
            V {width_mm - notch_height}
            H {x + notch_width}
            ""

        if x < width_mm / 2:
            circles_svg += f""
                <circle cx="{x + notch_width / 2 - 10}"
                    cy="{width_mm - notch_height / 2}"
                    r="{screw_d / 2}" />

```

```

        """
elif x > width_mm / 2:
    circles_svg += f"""
        <circle cx="{x + notch_width / 2 + 10}"
            cy="{width_mm - notch_height / 2}"
            r="{screw_d / 2}" />
        """
else:
    path_d += f"""
M {x} {width_mm}
V {width_mm - notch_height}
H {x + (notch_width - screw_d) / 2}
V {width_mm - (notch_height + nut_in)}
H {x + (notch_width - nut_w) / 2}
V {width_mm - (notch_height + nut_in + nut_l)}
H {x + (notch_width - screw_d) / 2}
V {width_mm - notch_width}
H {x + (notch_width + screw_d) / 2}
V {width_mm - (notch_height + nut_in + nut_l)}
H {x + (notch_width + nut_w) / 2}
V {width_mm - (notch_height + nut_in)}
H {x + (notch_width + screw_d) / 2}
V {width_mm - notch_height}
H {x + notch_width}
V {width_mm}
"""

if x < width_mm / 2:
    circles_svg += f"""
        <circle cx="{x + notch_width / 2 + 10}"
            cy="{width_mm - notch_height / 2}"
            r="{screw_d / 2}" />
        """

    circles_svg += f"""
        <circle cx="{x + notch_width / 2 - 10}"
            cy="{width_mm - notch_height / 2}"
            r="{screw_d / 2}" />
        """

elif x > width_mm / 2:
    # 右边缺口的凹槽
    circles_svg += f"""
        <circle cx="{x + notch_width / 2 - 10}"
            cy="{width_mm - notch_height / 2}"
            r="{screw_d / 2}" />
    """

```

```

'''
circles_svg += f'''
    <circle cx="{x + notch_width / 2 + 10}"
        cy="{width_mm - notch_height / 2}"
        r="{screw_d / 2}" />
'''

if width_mm < 95:
    path_d += f'M 20 0 V {notch_height}'
    path_d += f'M {width_mm - 20} 0 V {notch_height}'
    path_d += f'M 20 {width_mm} V {width_mm - notch_height}'
    path_d += f'M {width_mm - 20} {width_mm} V {width_mm - notch_height}'

# side rect
y = 10
step = notch_width + 10

while y < height_mm:
    if y < height_mm - 10:
        path_d += f''''
            M 0 {y}
            H {notch_height}
            V {y + (notch_width - screw_d)/2}
            H {notch_height + nut_in}
            V {y + (notch_width - nut_w)/2}
            H {notch_height + nut_in + nut_l}
            V {y + (notch_width - screw_d)/2}
            H {notch_width}
            V {y + (notch_width + screw_d)/2}
            H {notch_height + nut_in + nut_l}
            V {y + (notch_width + nut_w)/2}
            H {notch_height + nut_in}
            V {y + (notch_width + screw_d)/2}
            H {notch_height}
            V {y + notch_width}
            H 0
        ''''
    circles_svg += f'''
        <circle cx="{notch_height / 2}"
            cy="{y + notch_width / 2 + 10}"
            r="{screw_d / 2}" />
'''

    path_d += f''''

```

```

        M {width_mm} {y}
        H {width_mm - notch_height}
        V {y + (notch_width - screw_d) / 2}
        H {width_mm - (notch_height + nut_in)}
        V {y + (notch_width - nut_w) / 2}
        H {width_mm - (notch_height + nut_in + nut_l)}
        V {y + (notch_width - screw_d) / 2}
        H {width_mm - notch_width}
        V {y + (notch_width + screw_d) / 2}
        H {width_mm - (notch_height + nut_in + nut_l)}
        V {y + (notch_width + nut_w) / 2}
        H {width_mm - (notch_height + nut_in)}
        V {y + (notch_width + screw_d) / 2}
        H {width_mm - notch_height}
        V {y + notch_width}
        H {width_mm}
    """
    circles_svg += f"""
        <circle cx="{width_mm - notch_height / 2}"
              cy="{y + notch_width / 2 + 10}"
              r="{screw_d / 2}" />
    """

elif y == height_mm - 10:
    path_d += f"""
        M 0 {y}
        H {notch_height}
        V {y + notch_width - notch_height}
        H {notch_width}
        V {width_mm}
    """
    path_d += f"""
        M {width_mm} {y}
        H {width_mm - notch_height}
        V {y + notch_width - notch_height}
        H {width_mm - notch_width}
        V {width_mm}
    """
else:
    path_d += f"""
        M 0 {y}
        H {notch_height}
        V {width_mm}
    """
    path_d += f"""

```

```

        M {width_mm} {y}
        H {width_mm - notch_height}
        V {width_mm}
        """"
    y += step

#hollow king image
embedded_svg = f'''
<g>
    <path d="M17.375 4.25 C17.91125 4.4975 18.4475 4.745 19 5 C18.979375 5.763125 18.95875 6.52625
18.9375 7.3125 C18.87057159 10.10185786 18.87057159 10.10185786 20 12.999999999999998 C19.505
13.763125 19.01 14.52625 18.5 15.3125 C16.60362745 18.71016749 17.01665642 20.31246156 18 24 C18.33
24.66 18.66 25.32 19 26 C27.58 26 36.16 26 45 26 C47.15919484 18.69880511 47.15919484 18.69880511 44 13
C44.02025451 10.20994186 44.47669087 7.75926634 45 5 C46.625 4.25 46.625 4.25 49 4 C53.7606979
6.47953015 56.73603663 9.12377121 59 14 C60.01340792 21.43510506 59.50208906 26.95433754 55 33
C54.46890625 33.38414063 53.9378125 33.76828125 53.390625 34.1640625 C50.80106259 37.5828668
51.56413466 42.4179325 51.56054688 46.57617188 C51.42703895 51.8198107 50.54496554 54.88346197 47
59 C42.52950365 62.89524866 37.6331768 62.44847622 32 62.375 C31.11054688 62.38660156 30.22109375
62.39820313 29.3046875 62.41015625 C22.82495092 62.3811425 19.51953389 61.34755206 14.7109375
56.8671875 C12.34376797 52.90028696 12.57319232 49.65123379 12.625 45.125 C12.62737547 37.75505442
12.62737547 37.75505442 9 31.5625 C8.01 30.716875 7.02 29.87125 6 29 C4.0422079 22.97662369 4.00046547
16.26053591 6.675 10.5 C13.05 3.78947368 13.05 3.78947368 17.375 4.25 Z M14 8 C10.9332301 10.756442
8.40123603 13.31590049 7.42578125 17.41015625 C7.17011176 23.11423081 8.53714472 26.905834
11.7109375 31.6015625 C12.13632813 32.06304687 12.56171875 32.52453125 13 33 C13.66 33 14.32 33 15 33
C14.96519531 34.00160156 14.93039063 35.00320312 14.89453125 36.03515625 C14.45839979 45.96119362
14.45839979 45.96119362 18 55 C22.33312786 59.22202202 25.90460349 60.13835471 31.8125 60.25
C37.90191174 60.16928615 41.99493406 59.05783084 46.375 54.5625 C49.3077376 49.9377984 49.128386
45.61210687 49.0625 40.25 C49.05798828 39.55261719 49.05347656 38.85523437 49.04882812 38.13671875
C49.0371002 36.42444197 49.01916653 34.71220967 49 33 C49.99 32.67 50.98 32.34 52 32 C55.54289451
27.2423988 56.73899148 23.38865593 56.5390625 17.4453125 C55.62760816 13.31074427 53.0803554
10.76865277 50 8 C48.88267977 9.84334722 48.88267977 9.84334722 48 12 C48.309375 12.928125 48.61875
13.85625 48.9375 14.8125 C50.23609748 18.70829243 49.93911815 21.02680781 49 25 C48.03594077
26.68771075 47.0409554 28.35860132 46 30 C45.07960937 29.84402344 44.15921875 29.68804688
43.2109375 29.52734375 C34.17266167 28.13754777 26.13307741 27.95863216 17 29 C14.06832631
23.37118652 13.73326463 20.22811555 15 14 C15.33 13.34 15.66 12.68 16 12 C15.10836423 9.88316159
15.10836423 9.88316159 14 8 Z" fill="none" stroke="blue" stroke-width="0.05"/>    <path d="M0 0 C3 0.25 3
0.25 5 2.25 C5.265625 4.125 5.265625 4.125 5.25 6.25 C5.25515625 6.95125 5.2603125 7.6525 5.265625 8.375
C5 10.25 5 10.25 3 12.25 C0 12.5 0 12.5 -3 12.25 C-5 10.25 -5 10.25 -5.265625 8.375 C-5.26046875 7.67375
-5.2553125 6.9725 -5.25 6.25 C-5.25515625 5.54875 -5.2603125 4.8475 -5.265625 4.125 C-4.77431879
0.65695617 -3.28862917 0.27405243 0 0 Z" fill="none" stroke="blue" stroke-width="0.05"
transform="translate(41,40.75)"/>
    <path d="M0 0 C3 0.25 3 0.25 5 2.25 C5.265625 4.125 5.265625 4.125 5.25 6.25 C5.25515625 6.95125
5.2603125 7.6525 5.265625 8.375 C5 10.25 5 10.25 3 12.25 C0 12.5 0 12.5 -3 12.25 C-5 10.25 -5 10.25 -5.265625

```

```

8.375 C-5.26046875 7.67375 -5.2553125 6.9725 -5.25 6.25 C-5.25515625 5.54875 -5.2603125 4.8475 -5.265625
4.125 C-4.77431879 0.65695617 -3.28862917 0.27405243 0 0 Z" fill="none" stroke="blue" stroke-width="0.05"
transform="translate(23,40.75)"/>

```

```

</g>'''

```

```

    svg_content = f'''<?xml version="1.0" standalone="no"?>
<svg xmlns="http://www.w3.org/2000/svg"
    width="{width_in}in"
    height="{height_in}in"
    viewBox="0 0 {width_mm} {height_mm * 1.2}">

```

```

    <path d="{path_d}"
        fill="none"
        stroke="black"
        stroke-width="0.05"
        fill-rule="evenodd"/>

```

```

    <g fill="none" stroke="green" stroke-width="0.05">
        {circles_svg}
    </g>

```

```

    <g transform="
        translate({center_x}, {center_y})
        scale({scale})
        translate({-embed_w/2}, {-embed_h/2})"
    >
        {embedded_svg}
    </g>
</svg>
'''

```

```

    with open(filename, "w", encoding="utf-8") as f:
        f.write(svg_content)

```

```

    print(f"SVG file '{filename}' generated successfully.")
if __name__ == "__main__":
    try:
        w = float(input("Enter rectangle width (millimeters): "))
        generate_Part2_svg(w)
    except ValueError as e:
        print("Error:", e)

```

3. Top plane(part3) - Drawer Box Part 3.py

```
def generate_Part3_svg(width_mm, height_mm, filename="Drawer Box part3.svg"):
    if width_mm <= 60 or height_mm <= 60:
        raise ValueError("Width and height must be more than 60mm.")

    if width_mm > 150 or height_mm > 150:
        raise ValueError("Width and height must be 150mm or smaller.")

    notch_height = 4
    notch_width = 10
    screw_d = 2.3
    screw_l = 10
    screw_in = screw_l - notch_height
    nut_l = 1.8
    nut_w = 5
    nut_in = 3
    text_x = width_mm / 2
    text_y = height_mm / 3
    # real height is 2/3
    real_height = int(height_mm * 2 / 3)

    if width_mm < 95:
        notch_positions = [10, width_mm - 20]
    else:
        notch_positions = [10, 30, width_mm - 40, width_mm - 20]

    width_in = width_mm / 25.4
    height_in = real_height / 25.4
    path_d = ""
    circles_svg = ""

    # ===== SVG Path =====
    current_x = notch_height
    for x in notch_positions:
        if x > current_x:
            path_d += f"M {current_x} 0 H {x}\n"
            current_x = x + notch_width

    if current_x < width_mm:
        path_d += f"M {current_x} 0 H {width_mm - notch_height}\n"

    y = 0
    step = notch_width + 10
```

```

# side lines
while y + 10 < real_height - real_height % 20:
    path_d += f"M 0 {y + 10} V {min(y + 20, real_height)}\n"
    path_d += f"M {width_mm} {y + 10} V {min(y + 20, real_height)}\n"
    y += step

# down line
path_d += f"M {notch_height} {real_height} H {width_mm - notch_height}\n"

# up rect
for x in notch_positions:
    path_d += f""
    M {x} 0
    V {notch_height}
    H {x + (notch_width - screw_d) / 2}
    V {notch_height + nut_in}
    H {x + (notch_width - nut_w) / 2}
    V {notch_height + nut_in + nut_l}
    H {x + (notch_width - screw_d) / 2}
    V {notch_width}
    H {x + (notch_width + screw_d) / 2}
    V {notch_height + nut_in + nut_l}
    H {x + (notch_width + nut_w) / 2}
    V {notch_height + nut_in}
    H {x + (notch_width + screw_d) / 2}
    V {notch_height}
    H {x + notch_width}
    V {0}
    ""

    if x < width_mm / 2:
        circles_svg += f""
        <circle cx="{x + notch_width / 2 + 10}"
        cy="{notch_height / 2}"
        r="{screw_d / 2}" />
        ""

    elif x > width_mm / 2:
        circles_svg += f""
        <circle cx="{x + notch_width / 2 - 10}"
        cy="{notch_height / 2}"
        r="{screw_d / 2}" />
        ""

# side rect

```

```

y = 0
step = notch_width + 10

while y < real_height:
    if y + notch_width <= 20:
        path_d += f""
        M {notch_height} {y}
        V {y + notch_width}
        H 0
        ""

    elif y + notch_width <= real_height - 10:
        path_d += f""
        M 0 {y}
        H {notch_height}
        V {y + (notch_width - screw_d) / 2}
        H {notch_height + nut_in}
        V {y + (notch_width - nut_w) / 2}
        H {notch_height + nut_in + nut_l}
        V {y + (notch_width - screw_d) / 2}
        H {notch_width}
        V {y + (notch_width + screw_d) / 2}
        H {notch_height + nut_in + nut_l}
        V {y + (notch_width + nut_w) / 2}
        H {notch_height + nut_in}
        V {y + (notch_width + screw_d) / 2}
        H {notch_height}
        V {y + notch_width}
        H 0
        ""

    else :
        path_d += f""
        M 0 {y}
        H {notch_height}
        V {real_height}
        ""

    if y + notch_width <= 20:
        path_d += f""
        M {width_mm - notch_height} {y}
        V {y + notch_width}
        H {width_mm}
        ""

    elif y + notch_width <= real_height - 10:
        path_d += f""
        M {width_mm} {y}

```

```

        H {width_mm - notch_height}
        V {y + (notch_width - screw_d) / 2}
        H {width_mm - (notch_height + nut_in)}
        V {y + (notch_width - nut_w) / 2}
        H {width_mm - (notch_height + nut_in + nut_l)}
        V {y + (notch_width - screw_d) / 2}
        H {width_mm - notch_width}
        V {y + (notch_width + screw_d) / 2}
        H {width_mm - (notch_height + nut_in + nut_l)}
        V {y + (notch_width + nut_w) / 2}
        H {width_mm - (notch_height + nut_in)}
        V {y + (notch_width + screw_d) / 2}
        H {width_mm - notch_height}
        V {y + notch_width}
        H {width_mm}
        """"

    else :
        path_d += f""""
            M {width_mm} {y}
            H {width_mm - notch_height}
            V {real_height}
            """"

    y += step

svg_content = f'<?xml version="1.0" standalone="no"?>
<svg xmlns="http://www.w3.org/2000/svg"
    width="{width_in}in"
    height="{height_in}in"
    viewBox="0 0 {width_mm} {real_height}">

<path d="{path_d}"
    fill="none"
    stroke="black"
    stroke-width="0.05"
    fill-rule="evenodd"/>

<g fill="none" stroke="green" stroke-width="0.05">
    {circles_svg}
</g>
<text x="{text_x}"
    y="{text_y}"
    text-anchor="middle"
    font-family="Arial"

```

```

        font-size="{width_mm / 100 * 14}"
        fill="red">
    Drawer Box
</text>
</svg>
'''

with open(filename, "w", encoding="utf-8") as f:
    f.write(svg_content)

print(f"SVG file '{filename}' generated successfully.")

if __name__ == "__main__":
    try:
        w = float(input("Enter rectangle width (millimeters): "))
        h = float(input("Enter rectangle height (millimeters): "))

        generate_Part3_svg(w, h)

    except ValueError as e:
        print("Error:", e)

```

4. Lid(part4) - Drawer Box Part 4.py

```
def generate_Part4_svg(width_mm, height_mm, filename="Drawer Box part4.svg"):
    if width_mm <= 60 or height_mm <= 60:
        raise ValueError("Width and height must be more than 60mm.")
    if width_mm > 150 or height_mm > 150:
        raise ValueError("Width and height must be 150mm or smaller.")

    real_height = ((height_mm * 0.25) ** 2 + width_mm ** 2) ** 0.5 + 5 # 不再额外增加高度

    notch_height = 4
    rect_width = width_mm - notch_height - 2
    rect_height = real_height

    rect_x = (width_mm - rect_width) / 2
    rect_y = 0

    # ===== CU image parameters =====
    cu_w = 1264
    cu_h = 1028

    width_scale = (rect_width * 0.6) / cu_w
    height_scale = (rect_height * 0.6) / cu_h
    scale = min(width_scale, height_scale)

    image_w = cu_w * scale
    image_h = cu_h * scale

    image_x = rect_x + (rect_width - image_w) / 2
    image_y = rect_y + (rect_height - image_h) / 2

    # text parameters
    text_x = width_mm / 2
    text_y = rect_height * 0.85

    # ===== CU path =====
    embedded_svg = f"""
    <g transform="translate({image_x:.3f},{image_y:.3f}) scale({scale:.6f})">
    <path d="m620 4-14 1-2 1-2 54-51 1-5 2-3 9-1 5v32l3 11 4 3 13 1h40l2 9 1 11v22l-2 10-4 6-12 9-10 9-5 5-8
    9-8 13-7 16-4 14-1 8v12l4 16 4 10 11 18 10 14 10 10v2l-9 7-11 14-10
    7-10-1-39-9-52-14-28-7-49-16-35-11-30-8-30-5-51-7-11-1h-49l-30 5-29 7-14 6-17 12-10 8-10 9-14 14-9 13-8
    16-10 30-5 26v42l5 27 11 44 8 19 12 22 9 19 4 9 24 40 13 24 17 28 9 16 6 15 14 30 7 19 6 21 4 35 3 31 1
    1h933l1-1 2-24 5-33 8-36 6-16 16-32 13-27 14-24 9-14 6-9 6-14 10-17 11-17 7-14 6-15 9-17 12-29 6-23 4-30
    2-10v-33l-6-43-5-14-9-17-8-12-11-13-13-13-14-11-14-10-15-7-21-6-26-5-13-2h-49l-47 6-40 6-25 6-26 8-49 16-31
    """
```

```

9-36 9-28 8-44 10-11 1-12-12-6-7-9-7-3-2-1-3 5-2 12-11 7-11 8-16 4-9
4-14v-20l-5-19-8-19-7-12-9-9-11-9-15-13-4-5-2-6-1-8v-11l2-18 3-6v-2h46l6-2 6-7
2-7v-28l-3-13-2-4-5-2-11-1h-39l-1-1-1-43-1-9-3-2z" fill="none" stroke="blue" stroke-width="1"/>
    <path d="m1034 351h39l16 2 18 4 20 7 13 7 14 11 14 14 9 13 4 13 4 27v26l-5 40-5 17-9 21-10 20-6 10-6
13-8 16-11 18-7 11-14 26-13 17-7 11-9 15-7 9-11 13-10 13-10 14-4 5-11 9-13 13-8 7-15 11-17 9-13
3h-22l-13-4-8-7-4-6-2-8v-26l4-16 6-11 8-7 43-2 25-3 7-3 5-8 2-9 2-32-52 2h-21l-2-1 1-10 7-31
4-15v-4l-4-1h-32l-21 4-3 3-6 18-8 37-1 1-66 1-7 19-6 22-2 12 63 1 5 4v9l-5 15-14 21-8 10-10 9-14 9-10 5-19
3h-10l-13-2-12-6-10-9-12-14-11-15-8-16-10-31-4-16-1-7v-31l3-26 8-41 8-35 7-19 8-16 8-17 6-16 9-32
2-10v-13l9-6 22-12 23-11 20-8 22-8 28-7 61-19 43-12 37-8 12-2z" fill="none" stroke="blue" stroke-width="1"/>
    <path d="m189 351h40l24 3 30 6 45 12 45 14 52 15 26 9 36 16 23 13 8 5 1 4v13l4 17 16 44 14 30 7 20 6 21 5
30 6 41 1 16v14l-2 16-8 35-5 13-7 12-9 12-11 14-7 8-13 8-12 4-7
1h-14l-14-3-17-8-12-9-10-11-8-11-9-13-4-8-2-6v-11l4-4
2-1h50l10-2v-8l-8-35-3-9-63-1-4-4-5-16-9-30-4-8-8-3-12-2h-32l-6 1 1 10 10 30 2 9v11l-4 1h-16l-56-2 3 23 5 24 1 3
8 2 21 3 15 1 31 1 8 7 7 14 4 15 1 16-2 10-6 11-8 7-10 4-4
1h-21l-15-4-17-9-13-10-15-14-13-12-8-9-8-11-14-17-11-14-10-16-12-17-9-13-14-26-14-21-9-17-8-16-15-26-10-23
-7-26-6-31-2-14v-14l2-16 5-19 4-11 3-9 7-8 12-12 14-10 10-6 15-6 14-4 15-3z" fill="none" stroke="blue"
stroke-width="1"/>
    <path d="m165 945h945v75l-567 1h-80l-306-1-1-1v-73z" fill="none" stroke="blue" stroke-width="1"/>
</g>'''

```

```

# ===== Final SVG =====
svg_content = f'''<?xml version="1.0" standalone="no"?>
<svg xmlns="http://www.w3.org/2000/svg"
    width="{width_mm}mm"
    height="{real_height}mm"
    viewBox="0 0 {width_mm} {real_height}">

```

```

    <rect x="{rect_x}"
        y="{rect_y}"
        width="{rect_width}"
        height="{rect_height}"
        fill="none"
        stroke="black"
        stroke-width="0.05"/>

```

```

{embedded_svg}

```

```

    <text x="{text_x}"
        y="{text_y}"
        text-anchor="middle"
        font-family="Arial"
        font-size="{width_mm * 0.09}"
        fill="red">

```

```

    Digital Manufacturing

```

```

</text>
</svg>
'''

with open(filename, "w", encoding="utf-8") as f:
    f.write(svg_content)

print(f"SVG file '{filename}' generated successfully.")

if __name__ == "__main__":
    try:
        w = float(input("Enter width (mm): "))
        h = float(input("Enter height (mm): "))
        generate_Part4_svg(w, h)
    except ValueError as e:
        print("Error:", e)

```

5. Divider(part5) - Drawer Box Part 5.py

```
def generate_Part5_svg(width_mm, height_mm, filename="Drawer Box part5.svg"):
```

```
    if width_mm <= 60 or height_mm <= 60:
```

```
        raise ValueError("Width and height must be more than 60mm.")
```

```
    if width_mm > 150 or height_mm > 150:
```

```
        raise ValueError("Width and height must be 150mm or smaller.")
```

```
    notch_height = 4
```

```
    notch_width = 10
```

```
    real_height = 0.7 * height_mm
```

```
    width_in = width_mm / 25.4
```

```
    height_in = real_height / 25.4
```

```
    step = notch_width + 10
```

```
    path_d = ""
```

```
    #side lines
```

```
    y = 10
```

```
    step = notch_width + 10
```

```
    path_d += f"M {notch_height} 0 H {width_mm - notch_height}\n"
```

```
    path_d += f"M {notch_height} {real_height} H {width_mm - notch_height}\n"
```

```
    while y < real_height - 10:
```

```
        path_d += f"M {width_mm} {y} V {y + notch_width}\n"
```

```
        path_d += f"M 0 {y} V {y + notch_width}\n"
```

```
        y += step
```

```
    y = 0
```

```
    while y < real_height - step:
```

```
        if y < 20:
```

```
            path_d += f""
```

```
            M {width_mm - notch_height} 0
```

```
            V {y + notch_width}
```

```
            H {width_mm}
```

```
            ""
```

```
            path_d += f""
```

```
            M {notch_height} 0
```

```
            V {y + notch_width}
```

```
            H 0
```

```
            ""
```

```
        elif y < real_height:
```

```
            path_d += f""
```

```

        M {width_mm} {y}
        H {width_mm - notch_height}
        V {y + notch_width}
        H {width_mm}
        """"

        path_d += f""""
        M 0 {y}
        H {notch_height}
        V {y + notch_width}
        H 0
        """"

    y += step
    path_d += f"M {width_mm} {y} H {width_mm - notch_height} V {real_height}\n"
    path_d += f"M 0 {y} H {notch_height} V {real_height}\n"

    svg_content = f'<?xml version="1.0" standalone="no"?>
<svg xmlns="http://www.w3.org/2000/svg"
    width="{width_in}in"
    height="{height_in}in"
    viewBox="0 0 {width_mm} {real_height}">

    <path d="{path_d}"
        fill="none"
        stroke="black"
        stroke-width="0.05"
        fill-rule="evenodd"/>

</svg>
'''

    with open(filename, "w", encoding="utf-8") as f:
        f.write(svg_content)

    print(f"SVG file '{filename}' generated successfully.")

if __name__ == "__main__":
    try:
        w = float(input("Enter rectangle width (millimeters): "))
        h = float(input("Enter rectangle height (millimeters): "))

        generate_Part5_svg(w, h)
    except ValueError as e:
        print("Error:", e)

```

6. Left plane(part6) - Drawer Box Part 6.py

```
def generate_Part6_svg(width_mm, height_mm, filename="Drawer Box part6.svg"):
    if width_mm <= 60 or height_mm <= 60:
        raise ValueError("Width and height must be more than 60mm.")

    if width_mm > 150 or height_mm > 150:
        raise ValueError("Width and height must be 150mm or smaller.")

    notch_height = 4
    notch_width = 10
    screw_d = 2.3
    screw_l = 10
    screw_in = screw_l - notch_height
    nut_l = 1.8
    nut_w = 5
    nut_in = 3
    t_notch = ((0.25 * height_mm) ** 2 + (width_mm - 10) ** 2) ** 0.5 * notch_height / (width_mm - 10)
    slot_path_d = ""
    width_in = width_mm / 25.4
    height_in = height_mm / 25.4
    path_d = ""
    circles_svg = ""
    text_x = height_mm / 3 * 2
    text_y = width_mm * 0.8
    # ===== SVG Path =====

    # left up line
    path_d += f"M {height_mm * 0.25} 0 L 0 {width_mm - 10} L 0 {width_mm - notch_height} H {height_mm %
10}"

    # left up gap
    slot_path_d += f""
    M 0.5 {width_mm - notch_height}
    L {(height_mm * 0.25) * (width_mm - notch_height) / (width_mm - 10) + 0.5} 0
    L {(height_mm * 0.25) * (width_mm - notch_height) / (width_mm - 10) + 0.5 + t_notch} 0
    L {0.5 + t_notch} {width_mm - notch_height}
    Z
    ""

    # down line
    x = height_mm
    step = notch_width + 10
    while x > 10:
        path_d += f"M {x} {width_mm} H {max(x - 10, 10)}\n"
        x -= step
```

```

# make up down line gap
path_d += f"M {x + 10} {width_mm} V {width_mm - notch_height} H 0\n"
# up line
x = height_mm
n = 0
while x > height_mm * 0.25:
    if x > height_mm - notch_height:
        path_d += f"M {x - notch_height} 0 H {max(x - 10, height_mm * 0.25)}\n"
        n += 1
    elif x < height_mm:
        path_d += f"M {x} 0 H {max(x - 10, height_mm * 0.25)}\n"
        n += 1
    x -= step
# make up up gap
path_d += f"M {height_mm * 0.25} 0 H {height_mm - n * 20 + 30}\n"
# make up right up line
path_d += f"M {height_mm - notch_height} 0 V {notch_width} H {height_mm}\n"
# right line
y = 0
step = notch_width + 10
while y + 10 < width_mm:
    path_d += f"M {height_mm} {y + 10} V {min(y + 20, width_mm)}\n"
    y += step
# make up down right gap
path_d += f"M {height_mm} {y - 10} V {width_mm}\n"

# up rect
x = height_mm - notch_width
while x > height_mm * 0.25 + 20:
    path_d += f"M {x} 0 V {notch_height} H {x - notch_width} V 0\n"
    x -= step
# up hole
x = height_mm - notch_width - 10
while x > height_mm * 0.25 + 5:
    circles_svg += f'''
        <circle cx="{x - notch_width / 2}"
            cy="{notch_height / 2}"
            r="{screw_d / 2}"/>
        '''
    x -= step

# side rect + hole
y = 0
while y < width_mm:

```

```

if y + notch_width <= 20:
    path_d += f""
    M {height_mm - notch_height} {y}
    V {y + notch_width}
    H {height_mm}
    ""

elif y + notch_width <= width_mm - 20:
    path_d += f""
    M {height_mm} {y}
    H {height_mm - notch_height}
    V {y + (notch_width - screw_d) / 2}
    H {height_mm - (notch_height + nut_in)}
    V {y + (notch_width - nut_w) / 2}
    H {height_mm - (notch_height + nut_in + nut_l)}
    V {y + (notch_width - screw_d) / 2}
    H {height_mm - notch_width}
    V {y + (notch_width + screw_d) / 2}
    H {height_mm - (notch_height + nut_in + nut_l)}
    V {y + (notch_width + nut_w) / 2}
    H {height_mm - (notch_height + nut_in)}
    V {y + (notch_width + screw_d) / 2}
    H {height_mm - notch_height}
    V {y + notch_width}
    H {height_mm}
    ""

elif y + notch_width <= width_mm:
    path_d += f""
    M {height_mm} {y}
    H {height_mm - notch_height}
    V {y + notch_width}
    H {height_mm}
    ""

circles_svg += f""
    <circle cx="{height_mm - notch_height / 2}"
    cy="{y + notch_width / 2 + 10}"
    r="{screw_d / 2}" />
    ""

y += step

# down rect + hole
x = height_mm
while x > 10:
    current_x = x + 10

```

```

if current_x - notch_width >= height_mm - 10:
    path_d += f""
    M {current_x} {width_mm}
    H {current_x - notch_width}
    V {width_mm - notch_height}
    ""

elif current_x - notch_width >= 10:
    path_d += f""
    M {current_x} {width_mm}
    V {width_mm - notch_height}
    H {current_x - (notch_width - screw_d) / 2}
    V {width_mm - (notch_height + nut_in)}
    H {current_x - (notch_width - nut_w) / 2}
    V {width_mm - (notch_height + nut_in + nut_l)}
    H {current_x - (notch_width - screw_d) / 2}
    V {width_mm - notch_width}
    H {current_x - (notch_width + screw_d) / 2}
    V {width_mm - (notch_height + nut_in + nut_l)}
    H {current_x - (notch_width + nut_w) / 2}
    V {width_mm - (notch_height + nut_in)}
    H {current_x - (notch_width + screw_d) / 2}
    V {width_mm - notch_height}
    H {current_x - notch_width}
    V {width_mm}
    ""

if x >= 20 and x < height_mm - 10:
    circles_svg += f""
    <circle cx="{x - notch_width / 2}"
    cy="{width_mm - notch_height / 2}"
    r="{screw_d / 2}" />
    ""

x -= step

# middle connection
x = height_mm
x1 = width_mm * 1 / 3 - notch_height
x2 = width_mm * 2 / 3 - notch_height
while x - notch_height > 0.25 * height_mm + 25:
    path_d += f"M {x - 10 - notch_height} {x1} V {x1 + notch_height} H {x - 20 - notch_height} V {x1} Z\n"
    path_d += f"M {x - 10 - notch_height} {x2} V {x2 + notch_height} H {x - 20 - notch_height} V {x2} Z\n"
    x -= step

```

```

svg_content = f'<?xml version="1.0" standalone="no"?>
<svg xmlns="http://www.w3.org/2000/svg"
width="{height_in}in"
height="{width_in}in"
viewBox="0 0 {height_mm} {width_mm}">
<path d="{path_d}"
fill="none"
stroke="black"
stroke-width="0.05"/>
<path d="{slot_path_d}"
fill="none"
stroke="red"
stroke-width="0.05"/>

<g fill="none" stroke="green" stroke-width="0.05">
{circles_svg}
</g>
<text x="{text_x}"
y="{text_y}"
text-anchor="middle"
font-family="Arial"
font-size="{height_mm * 0.08}"
fill="red"
transform="scale(-1, 1) translate({-text_x * 2}, 0)">
ws2782
</text>
</svg>
'''

```

with open(filename, "w", encoding="utf-8") as f:

f.write(svg_content)

print(f"SVG file '{filename}' generated successfully.")

if __name__ == "__main__":

try:

w = float(input("Enter rectangle width (millimeters): "))

h = float(input("Enter rectangle height (millimeters): "))

generate_Part6_svg(w, h)

except ValueError as e:

print("Error:", e)

7. Right plane(part7) - Drawer Box Part 7.py

```
def generate_Part7_svg(width_mm, height_mm, filename="Drawer Box part7.svg"):
    if width_mm <= 60 or height_mm <= 60:
        raise ValueError("Width and height must be more than 60mm.")

    if width_mm > 150 or height_mm > 150:
        raise ValueError("Width and height must be 150mm or smaller.")

    notch_height = 4
    notch_width = 10
    screw_d = 2.3
    screw_l = 10
    screw_in = screw_l - notch_height
    nut_l = 1.8
    nut_w = 5
    nut_in = 3
    t_notch = ((0.25 * height_mm) ** 2 + (width_mm - 10) ** 2) ** 0.5 * notch_height / (width_mm - 10)
    slot_path_d = ""
    width_in = width_mm / 25.4
    height_in = height_mm / 25.4
    path_d = ""
    circles_svg = ""
    text_x = height_mm / 3 * 2
    text_y = width_mm * 0.8
    # ===== SVG Path =====

    # left up line
    path_d += f"M {height_mm * 0.25} 0 L 0 {width_mm - 10} L 0 {width_mm - notch_height} H {height_mm %
10}"

    # left up gap
    slot_path_d += f""
    M 0.5 {width_mm - notch_height}
    L {(height_mm * 0.25) * (width_mm - notch_height) / (width_mm - 10) + 0.5} 0
    L {(height_mm * 0.25) * (width_mm - notch_height) / (width_mm - 10) + 0.5 + t_notch} 0
    L {0.5 + t_notch} {width_mm - notch_height}
    Z
    ""

    # down line
    x = height_mm
    step = notch_width + 10
    while x > 10:
        path_d += f"M {x} {width_mm} H {max(x - 10, 10)}\n"
        x -= step
```

```

# make up down gap
path_d += f"M {x + 10} {width_mm} V {width_mm - notch_height} H 0\n"
# up line
x = height_mm
n = 0
while x > height_mm * 0.25:
    if x > height_mm - notch_height:
        path_d += f"M {x - notch_height} 0 H {max(x - 10, height_mm * 0.25)}\n"
        n += 1
    elif x < height_mm:
        path_d += f"M {x} 0 H {max(x - 10, height_mm * 0.25)}\n"
        n += 1
    x -= step
# make up up left gap
path_d += f"M {height_mm * 0.25} 0 H {height_mm - n * 20 + 30}\n"
# make up up right gap
path_d += f"M {height_mm - notch_height} 0 V {notch_width} H {height_mm}\n"
# right line
y = 0
step = notch_width + 10
while y + 10 < width_mm:
    path_d += f"M {height_mm} {y + 10} V {min(y + 20, width_mm)}\n"
    y += step
# make up right down gap
path_d += f"M {height_mm} {y - 10} V {width_mm}\n"

# up rect
x = height_mm - notch_width
while x > height_mm * 0.25 + 20:
    path_d += f"M {x} 0 V {notch_height} H {x - notch_width} V 0\n"
    x -= step
# up hole
x = height_mm - notch_width - 10
while x > height_mm * 0.25 + 5:
    circles_svg += f'''
        <circle cx="{x - notch_width / 2}"
            cy="{notch_height / 2}"
            r="{screw_d / 2}"/>
        '''
    x -= step

# right rect + hole
y = 0
while y < width_mm:

```

```

if y + notch_width <= 20:
    path_d += f""
    M {height_mm - notch_height} {y}
    V {y + notch_width}
    H {height_mm}
    ""

elif y + notch_width <= width_mm - 20:
    path_d += f""
    M {height_mm} {y}
    H {height_mm - notch_height}
    V {y + (notch_width - screw_d) / 2}
    H {height_mm - (notch_height + nut_in)}
    V {y + (notch_width - nut_w) / 2}
    H {height_mm - (notch_height + nut_in + nut_l)}
    V {y + (notch_width - screw_d) / 2}
    H {height_mm - notch_width}
    V {y + (notch_width + screw_d) / 2}
    H {height_mm - (notch_height + nut_in + nut_l)}
    V {y + (notch_width + nut_w) / 2}
    H {height_mm - (notch_height + nut_in)}
    V {y + (notch_width + screw_d) / 2}
    H {height_mm - notch_height}
    V {y + notch_width}
    H {height_mm}
    ""

elif y + notch_width <= width_mm:
    path_d += f""
    M {height_mm} {y}
    H {height_mm - notch_height}
    V {y + notch_width}
    H {height_mm}
    ""

circles_svg += f""
    <circle cx="{height_mm - notch_height / 2}"
    cy="{y + notch_width / 2 + 10}"
    r="{screw_d / 2}" />
    ""

y += step

# down rect + hole
x = height_mm
while x > 10:
    current_x = x + 10

```

```

if current_x - notch_width >= height_mm - 10:
    path_d += f""
    M {current_x} {width_mm}
    H {current_x - notch_width}
    V {width_mm - notch_height}
    ""

elif current_x - notch_width >= 10:
    path_d += f""
    M {current_x} {width_mm}
    V {width_mm - notch_height}
    H {current_x - (notch_width - screw_d) / 2}
    V {width_mm - (notch_height + nut_in)}
    H {current_x - (notch_width - nut_w) / 2}
    V {width_mm - (notch_height + nut_in + nut_l)}
    H {current_x - (notch_width - screw_d) / 2}
    V {width_mm - notch_width}
    H {current_x - (notch_width + screw_d) / 2}
    V {width_mm - (notch_height + nut_in + nut_l)}
    H {current_x - (notch_width + nut_w) / 2}
    V {width_mm - (notch_height + nut_in)}
    H {current_x - (notch_width + screw_d) / 2}
    V {width_mm - notch_height}
    H {current_x - notch_width}
    V {width_mm}
    ""

if x >= 20 and x < height_mm - 10:
    circles_svg += f""
    <circle cx="{x - notch_width / 2}"
    cy="{width_mm - notch_height / 2}"
    r="{screw_d / 2}" />
    ""

x -= step

# middle connection
x = height_mm
x1 = width_mm * 1 / 3 - notch_height
x2 = width_mm * 2 / 3 - notch_height
while x - notch_height > 0.25 * height_mm + 25:
    path_d += f"M {x - 10 - notch_height} {x1} V {x1 + notch_height} H {x - 20 - notch_height} V {x1} Z\n"
    path_d += f"M {x - 10 - notch_height} {x2} V {x2 + notch_height} H {x - 20 - notch_height} V {x2} Z\n"
    x -= step

svg_content = f'<?xml version="1.0" standalone="no"?>

```

```

<svg xmlns="http://www.w3.org/2000/svg"
      width="{height_in}in"
      height="{width_in}in"
      viewBox="0 0 {height_mm} {width_mm}">
  <g transform="scale(-1, 1) translate(-{height_mm}, 0)">
    <path d="{path_d}"
          fill="none"
          stroke="black"
          stroke-width="0.05"/>
    <path d="{slot_path_d}"
          fill="none"
          stroke="red"
          stroke-width="0.05"/>

  <g fill="none" stroke="green" stroke-width="0.05">
    {circles_svg}
  </g>
</g>

  <text x="{text_x}"
        y="{text_y}"
        text-anchor="middle"
        font-family="Arial"
        font-size="{height_mm * 0.08}"
        fill="red"
        transform="scale(-1, 1) translate({-text_x * 2}, 0)">
    ws2782
  </text>
</svg>
'''

with open(filename, "w", encoding="utf-8") as f:
    f.write(svg_content)

    print(f'SVG file '{filename}' generated successfully.')

if __name__ == "__main__":
    try:
        w = float(input("Enter rectangle width (millimeters): "))
        h = float(input("Enter rectangle height (millimeters): "))

        generate_Part7_svg(w, h)

    except ValueError as e:
        print("Error:", e)

```

Grading rubrics reached

- 10pt Cover page correct and complete✓

P1

- 10pt small rectangle cut correctly (SVG+Photo) submitted a week before deadline✓

P2

- 10pt Software inputs length, width and height of box and detects errors (show input dialog)✓

P4-5

- 10pt A description of the software you wrote - calculation steps, formulas, conditions.✓

P10-13

- 20pt Four examples showing input parameters and output flat pattern.✓

P5-9

- 20pt Printout of the two SVG files and photo of boxes they produced✓

P2-3, 6-8

- 10pt User-specified text engraved on the top of the vessel✓

P2-3 Drawer Box in top plane

- 10pt User-specified text engraved on the front or side of the vessel✓

P2-3 ws2782 in side plane

- 10pt Columbia logo and “Digital Manufacturing” engraved on the

front of the vessel✓

P2-3 in lid

- 10pt A fractal engraving generated by the program embedded on one

side of the vessel✓

P2-3 snowflake in bottom plane

- 10pt a removable lid to cover the container✓

P2-3

- 10pt a divider that inserts into the container at a user-specified position

✓

P2-3

- 10pt Any additional feature accommodated by software, e.g. rounded

corners, slant angle✓

P2-3 lid notch

Total: 150 points